Ragionare sui programmi: induzione strutturale

L08 · Programmazione funzionale · OCaml

Mauro Leoncini

Obiettivi

- Dimostrare che una proprietà vale per **ogni** lista e per **ogni** albero
- Usare le equazioni del programma come assiomi, sapendo **perché** è lecito
- Scrivere la prova in un formato controllabile — quello dell'esame
- Riconoscere quando l'enunciato va **generalizzato**, e che cosa costa

Oggi si vede **una** dimostrazione fatta insieme, e poi se ne fanno quattro.

:::

Quattro "debiti" da saldare

da	l'enunciato
L04	`(xs @ ys) @ zs = xs @ (ys @ zs)`
L05	`size t = length (inorder t)`
L06	`map f (map g l) = map (g %> f) l`
L07	`rev_acc [] l = naive_rev l`

Li abbiamo **controllati** su qualche esempio. Oggi li **dimostriamo**: per ogni valore, non per quelli provati.

:::

Il piano di oggi, in quattro tempi

1. Che cos'è un assioma qui — e perché le clausole di una funzione si possono usare come equazioni. Dieci minuti: il minimo per poter scrivere la prima riga.

2. Un esempio guidato, che NON è uno dei quattro debiti:

```
length (xs @ ys) = length xs + length ys
```

Lo dimostriamo insieme alla lavagna. Lo scopo non è il risultato: è **fissare il formato** che poi userete voi.

3. Le domande che la dimostrazione avrà fatto nascere, e "dovrebbero" essere tre.

4. I quattro debiti, uno per tecnica: il primo lo saldate voi.

:::

La tecnica di base

Ogni lista è costruita con un numero **finito** di `::` a partire da `[]`.

Quindi, per dimostrare che una proprietà P vale per ogni lista, basta:

- **base**: P vale per `[]`
- **passo**: per ogni `x` e `r`, se P vale per `r`, allora vale per `x :: r`

È l'induzione sui naturali, con `[]` al posto di 0 e `x :: _` al posto del «successore».

:::

Il programma è l'assiomatizzazione di sé stesso

```
let rec append l1 l2 =
  match l1 with
  | [] -> l2
  | x :: r -> x :: append r l2
```

```
let rec length = function
  | [] -> 0
  | _ :: r -> 1 + length r
```

Una **notazione**: `append xs ys` si scrive anche `xs @ ys`. Stessa funzione, notazione infissa — come `+` invece di `add`.

```
[A1] [] @ ys       = ys
[A2] (x :: r) @ ys = x :: (r @ ys)
```

```
[L1] length [] = 0
[L2] length (x :: r) = 1 + length r
```

Ogni clausola è un'**equazione**. Queste quattro sono tutto ciò che serve per la dimostrazione guidata.

E `@`, da qui alla fine, significa **esattamente** «la funzione definita da `[A1]` e `[A2]`».

:::

Sostituire uguali con uguali

Ogni passaggio di una dimostrazione sostituisce un'espressione con una **uguale**, e le equazioni si usano **in entrambe le direzioni**.

È lecito perché le due parti di un'equazione **denotano lo stesso valore**, ovunque compaiano: è la trasparenza referenziale di L03.

In C++, `f(x) + f(x)` e `2 * f(x)` non sono intercambiabili se `f` ha effetti. Qui lo sono sempre.

:::

Perché una clausola è un'equazione

Il ramo dice un **passo di riduzione**, non un uguale:

`append (x :: r) ys  →  x :: append r ys`

L'uguale si guadagna con un fatto di L03: in un linguaggio puro, **la riduzione conserva il valore denotato**. Quindi `=` non significa «il programma riscrive l'uno nell'altro», significa «le due espressioni denotano lo stesso valore».

Ed è per questo che l'equazione si può usare **nei due versi**: l'uguale è simmetrico, la freccia no.

:::

Come si scrive, anche all'esame

1. l'**enunciato**, con i suoi quantificatori
2. su quale variabile si fa induzione, e **perché** quella
3. il **caso base**
4. il **passo**, con l'ipotesi induttiva scritta **per esteso** prima di usarla
5. ogni riga giustificata fra parentesi quadre

```
  length ((x :: r) @ ys)
= length (x :: (r @ ys))              [A2]
= 1 + length (r @ ys)                 [L2]
```

:::

Enunciato e variabile su cui si fa induzione

Enunciato

Per ogni `xs`, `ys`:

```
length (xs @ ys) = length xs + length ys
```

Su che cosa si fa induzione? Su `xs` o su `ys`?

`append` **ricorre sul primo argomento**: `ys` viaggia intatto fino in fondo. L'equazione `[A2]` si applica solo quando il primo argomento è della forma `x :: r`.

Si fa induzione sull'argomento su cui la funzione ricorre. La prova segue il programma.

:::

Il caso base

```
Base, xs = []:
    length ([] @ ys)
  = length ys                         [A1]
  = 0 + length ys                     [aritmetica]
  = length [] + length ys             [L1, da destra a sinistra]
```

:::

Il passo induttivo con l'ipotesi

```
Passo, xs = x :: r.  II: length (r @ ys) = length r + length ys
    length ((x :: r) @ ys)
  = length (x :: (r @ ys))            [A2]
  = 1 + length (r @ ys)               [L2]
  = 1 + (length r + length ys)        [II]
  = (1 + length r) + length ys        [associatività di +]
  = length (x :: r) + length ys       [L2, da destra a sinistra]
```

Tre domande che nascono da quella prova

La prima arriva sempre, e suona così: *«ma allora `@` è la concatenazione, e stiamo dimostrando qualcosa che già sapevamo?»*

Le equazioni sono la definizione

Non sono affermazioni su una concatenazione che già conoscete, da verificare. Nelle dimostrazioni `@` significa **esattamente** «la funzione definita da `[A1]` e `[A2]`».

Cambiate il corpo e cambiate la funzione, e l'equazione cambia con lei, restando vera:

```
| x :: r -> f r l2        (*  f (x :: r) ys = f r ys  è VERA  *)
```

:::

Le equazioni sono calcolo, non induzione

Su una lista concreta bastano, e l'induzione non compare:

```
append [1;2] [3]
→ 1 :: append [2] [3]          [A2]
→ 1 :: (2 :: append [] [3])    [A2]
→ 1 :: (2 :: [3])              [A1]
=  [1;2;3]
```

Tre passi, nessuna ipotesi. L'ipotesi induttiva dice un'altra cosa: afferma qualcosa **sul risultato** di una chiamata che non è stata calcolata.

> `[A2]` e `[L2]` smontano un'espressione di un passo. `[II]` è l'unico momento in cui si **sa** qualcosa di non calcolato.

:::

Quando una clausola non è un'equazione

```
let classify = function
  | x :: _ when x > 0 -> "pos"
  | _ :: _            -> "nonpos"
  | [] -> "empty"
```

Il secondo ramo, letto come equazione, direbbe `classify (x :: r) = "nonpos"` per ogni `x`. Ma `classify [5] = "pos"`.

> Le clausole sono equazioni **quando i pattern sono disgiunti**. Altrimenti sono equazioni **condizionate**, e la condizione va scritta.

:::

Il foglio delle equazioni

```
[A1]  [] @ ys = ys                    [R1]  naive_rev [] = []
[A2]  (x::r) @ ys = x :: (r @ ys)     [R2]  naive_rev (x::r) = naive_rev r @ [x]
```

```
[L1]  length [] = 0
[L2]  length (x::r) = 1 + length r

[M1]  map f [] = []
[M2]  map f (x::r) = f x :: map f r

[C]   (g %> f) x = f (g x)
```

```
[RA1] rev_acc acc [] = acc
[RA2] rev_acc acc (x::r)
      = rev_acc (x::acc) r

[D1]  size Leaf = 0
[D2]  size (Node (l,x,r))
      = size l + 1 + size r

[I1]  inorder Leaf = []
[I2]  inorder (Node (l,x,r))
      = inorder l @ ([x] @ inorder r)
```

Sono le clausole dei programmi di L04–L07, una per riga. Niente altro.

:::

Ora voi: il debito di L04

Per ogni `xs`, `ys`, `zs`:

```
(xs @ ys) @ zs = xs @ (ys @ zs)
```

Quindici minuti, nel formato di prima.

:::

Le leggi di L06

```
length (map f l) = length l
```

```
map f (map g l) = map (g %> f) l
```

Nella prima, `f` non compare mai nel ragionamento: la proprietà vale per **ogni** `f`. È la parametricità di L06, **dimostrata** invece che letta nel tipo.

:::

Alberi: due ipotesi induttive

```
type 'a tree = Leaf | Node of 'a tree * 'a * 'a tree
```

- **base**: P vale per `Leaf`
- **passo**: se P vale per `l` **e** per `r`, allora vale per `Node (l, x, r)`

Il costruttore ha **due** occorrenze ricorsive, e il passo ha **due** ipotesi.

```
size t = length (inorder t)
```

:::

Un caso per costruttore, tre volte

La promessa di L04, mantenuta:

	lista	albero
tipo	`[]` · `x :: r`	`Leaf` · `Node (l, x, r)`
funzione	un caso per costruttore	un caso per costruttore
prova	base · passo con 1 ipotesi	base · passo con 2 ipotesi

La forma del tipo detta la forma della funzione, e detta la forma della dimostrazione.

Il debito di L07

```
rev_acc [] l = naive_rev l
```

Alla lavagna, il tentativo diretto. Il passo arriva a

`rev_acc [] (x :: r) = rev_acc [x] r` `[RA2]`

e si ferma. L'ipotesi parla di `rev_acc [] r`; noi abbiamo `rev_acc [x] r`.

:::

Generalizzare

Si dimostra un enunciato **più forte**:

$$\forall l.\ \forall acc.\ \texttt{rev_acc acc l = naive_rev l @ acc}$$

Il «per ogni `acc`» sta **dentro** l'enunciato su cui si fa induzione: così nel passo l'ipotesi si può usare con l'accumulatore `x :: acc`.

Poi, con `acc = []` e il Lemma `xs @ [] = xs`, il corollario.

:::

L'invariante è tornato

L'enunciato generalizzato **non viene dal tipo**: l'abbiamo inventato.

«Ciò che è già nell'accumulatore, più il rovesciamento di ciò che resta, dà il risultato»: è l'**invariante di un ciclo**, quello di L02.

L07 ha reso il programma più veloce. L08 mostra che lo ha reso **più difficile da dimostrare**: l'accumulatore trasforma la ricorsione in un ciclo, e con il ciclo torna il costo di inventarne l'invariante.

:::

Prima di dimostrare, controllare

```
T1  length (xs @ ys) = length xs + length ys   holds on 5000 cases
...
??  map f (xs @ ys) = map f ys @ map f xs      FALSE
```

`codice/leggi.ml` prova ogni enunciato di oggi su 5000 ingressi casuali.

Non è una dimostrazione. È il modo di non passare un'ora a dimostrare una cosa falsa.

:::

Che cosa l'induzione non copre

Valori infiniti. OCaml accetta

```
let rec cyclic = 1 :: 2 :: cyclic        (* [1; 2; <cycle>] *)
```

Non c'è un `[]` in fondo: `length cyclic` non restituisce nessun valore.

Effetti. Con `f` e `g` che stampano, le due parti della fusione danno gli stessi valori e stampano in ordine diverso:

```
map f (map g l):  g3 g2 g1 f4 f3 f2
map (g %> f) l:   g3 f4 g2 f3 g1 f2
```

:::

Che cosa portate a casa oggi

- Le clausole di una funzione sono **equazioni**, e bastano come assiomi
- Si sostituiscono uguali con uguali **perché** c'è la trasparenza referenziale
- Si fa induzione sull'argomento su cui la funzione ricorre
- Un caso per costruttore, un'ipotesi per ogni occorrenza ricorsiva
- Con un accumulatore l'enunciato va **generalizzato**: l'invariante torna
- Prima di dimostrare, controllare su molti casi

Al calcolatore

```
lezioni/L08-induzione/codice/
  definizioni.ml     le funzioni di oggi, un nome per ogni equazione
  dimostrazioni.md   le sette prove complete, riga per riga
  leggi.ml           ogni enunciato su 5000 casi, e uno falso
  limiti.ml          la lista ciclica e la fusione con gli effetti
  esercizi.ml        cinque esercizi; ogni enunciato già controllato
```

:::

Riferimenti

- Da fare prima di L09: esercizi 1 e 2, **scritti** nel formato di oggi
- Domanda per L09: `leggi.ml` ha trovato falsa l'ultima legge. Qual è il **più piccolo** ingresso che la rende falsa? Trovatelo a mano: a L09 lo chiederemo a QCheck.